perplexity

✅ Week 1 — Task 2: Nestup Branding Done

I've built the complete **Nestup Brand Kit** as an interactive HTML file (shareable with your team, opens in any browser). Here's everything that's inside:

What's in the Brand Kit

01 — Logo System

- Inline SVG logo mark — a minimal house icon with an **ochre dot** at the roof peak (the "found home" moment)
- 4 variants: Light bg, Dark mode, On-brand (on teal), and Icon-only (app icon)
- Wordmark in **Outfit 800** — clean, modern, slightly rounded

02 — Color System

Role	Color	Hex
Primary (Trust)	Forest Teal	`#1a6b5a`
Accent (Energy)	Warm Ochre	`#d97b3a`
Background	Stone Cream	`#f5f4f0`
Primary Text	Charcoal	`#1e1c17`
Muted Text	Slate Gray	`#6b6860`

03 — Splash Screen Mockup

- Teal gradient + subtle depth circles
- Logo, tagline ("Find your perfect **nestup**."), and category strip
- Full phone frame preview (300×620px)

04 — App Header

- Logo + wordmark left, avatar + notification bell right
- Ochre dot badge on bell
- Teal gradient hero band with a frosted-glass search bar below

05 — Listing Card Components

- FOR RENT / FOR SALE / PG badge variants
- Save (heart) button, price, location, and metadata tags

- Hover elevation effect built in

06 — **Icon Vocabulary**
16 core Lucide icons (Home, Search, Filter, Post, Save, Enquiry, etc.) — consistent 2px stroke throughout

How to Use This With Your App

- **Colors** → copy the hex values directly into your `ThemeData` (Flutter) or `StyleSheet` (React Native)
- **Logo SVG** → export as PNG at 2x/3x or use the SVG path in your Flutter `CustomPainter` / RN `react-native-svg`
- **Fonts** → use **Outfit** (display/headings) + **Plus Jakarta Sans** (body/UI) — both free on Google Fonts
- **Splash screen** → the gradient + logo layout maps 1:1 to a Flutter `SplashScreen` widget

Up Next → Task 3: Login / Register (Phone OTP)

Ready to build the **OTP login/register screens** whenever you say go. I'll cover:

- Phone number entry screen
- OTP input screen (6-digit, auto-advance)
- Error states (wrong OTP, resend timer)
- Skip/Guest mode option

Just say "**next**" and we'll jump straight into it!

next

Next, build the **Login / Register flow with phone OTP** as a simple 2-screen sequence: first collect the phone number, then verify a 6-digit code, with auto-read support when Firebase completes verification automatically and a resend timer for failures or delays. Keep the UI distraction-free, make errors explicit, and avoid overloading the first screen with extra actions so users can finish sign-in quickly.[1] [2] [3]

Recommended flow

Use this exact flow for Nestup: **Phone entry → OTP verify → success redirect to home/profile completion.** Firebase phone auth supports sending the code, returning a confirmation state, and auto-verifying on some Android devices, so your UI should be ready to skip manual OTP entry when verification completes in the background.[2] [3]

- Screen 1: Country code + phone number, primary CTA “Continue”, small helper text about OTP delivery.[4] [1]

- Screen 2: 6-digit OTP input, resend countdown, change number option, and clear invalid-code messaging.[5] [2]
- Success state: If Firebase auto-verifies, hide the OTP form and navigate forward immediately instead of letting the user enter a code that may already be consumed.[2]

Screen structure

For the **phone screen**, keep one clear primary action and only essential fields, because simpler login screens reduce friction and improve completion. For the **OTP screen**, use separated digit boxes or a single hidden input with 6 visible slots, auto-advance focus, auto-submit on 6 digits, and a disabled resend button until the timer ends.[6] [1]

Firebase logic

In React Native Firebase, the standard pattern is `signInWithPhoneNumber()` to request the SMS and `confirm.confirm(code)` to verify it. On Android or some supported environments, `onAuthStateChanged` may fire after automatic SMS handling, so your code should treat that as a successful login and move on. Firebase's official Android phone auth also supports callbacks for `onCodeSent`, `onVerificationCompleted`, and `onVerificationFailed`, which maps neatly to your UI states: sent, auto-verified, and error.[3] [5] [2]

Validation rules

Validate the number before sending the OTP and show a specific error instead of a generic failure message. For Indian users, prefill `+91` by default, accept only digits in the local field, and disable the CTA until the number length is valid. On OTP entry, show "Invalid OTP" or "Code expired, resend again" inline near the boxes instead of only using toast messages.[5] [1]

Nestup UI direction

Keep this flow visually aligned with the brand kit: Forest Teal as the main CTA color, warm stone backgrounds, and the Nestup logo at the top of both screens for continuity with the splash screen. The OTP screen should feel calmer than the home screen, with minimal chrome, one primary CTA, and a subtle "Change number" text action below the phone summary.[6] [1]

What I suggest building next

Build these in this order:

1. **Phone entry screen**
2. **OTP verification screen**
3. **States** — loading, invalid OTP, resend timer, auto-verified success
4. **Firebase wiring**
5. **Profile completion fallback** for first-time users after successful sign-in

If you want, I can now create the **actual Nestup OTP screen UI** for your stack. Which stack are you using: **Flutter** or **React Native**?

⁂

✅ Week 1 — Task 3: Login / Register (Phone OTP) Done

Here's everything in the zip — **7 production-ready files**, drop straight into your RN project.

What's inside

```
nestup-auth/
├── screens/auth/
│   ├── PhoneEntryScreen.jsx    ← Screen 1
│   ├── OTPVerifyScreen.jsx     ← Screen 2
│   └── ProfileSetupScreen.jsx  ← Screen 3 (new users only)
├── components/
│   └── NestupLogo.jsx          ← Reusable SVG logo (react-native-svg)
├── navigation/
│   └── AuthNavigator.jsx       ← Stack + useAuthGate() hook
├── theme.js                    ← All design tokens (matches brand kit)
└── README.md                   ← Full setup guide
```

Key features built

Feature	Detail
Phone screen	`+91` prefilled, 10-digit validation, CTA disabled until valid, inline errors
OTP screen	6 separate boxes, auto-advance focus, auto-submit on 6 digits, backspace handling
Android auto-verify	`onAuthStateChanged` listener skips manual entry when Firebase auto-reads SMS
Resend timer	30s countdown, resets timer on resend, new `confirmation` ref stored
Error states	Wrong OTP, expired OTP, rate-limited, no connection — all with specific messages
New vs returning user	`creationTime === lastSignInTime` check → ProfileSetup or Home
Profile setup	Name saved to `users/{uid}` in Firestore with phone + timestamp
Guest mode	"Browse as guest" bypasses auth entirely
`useAuthGate()`	Hook for App.tsx root — checks Firebase session on launch, shows splash during load

Install these deps first

```
npm install @react-native-firebase/app @react-native-firebase/auth \
  @react-native-firebase/firestore \
  @react-navigation/native @react-navigation/native-stack \
  react-native-svg react-native-screens react-native-safe-area-context
```

And in `App.tsx`:

```
const { user, loading } = useAuthGate();
if (loading) return <SplashScreen />;
return user ? <MainNavigator /> : <AuthNavigator />;
```

Week 1 progress

- [x] Fix all crashes
- [x] Nestup branding
- [x] Login / Register (Phone OTP) ← **just done**
- [ ] **Real listing cards with Firebase data** ← next up

Say "**next**" and I'll build the Firebase-powered listing cards with Firestore real-time data, skeleton loaders, and the full card component!

✅ Week 1 — Task 4: Real Listing Cards with Firebase Data Done

I’ve prepared a React Native package for the listings feed with realtime Firestore support, because `onSnapshot` is the standard RN Firebase pattern for keeping FlatList data synced automatically as documents change. The package also uses a document-per-listing Firestore structure, which matches Firebase’s collection/document model and fits real-estate listing data cleanly.[16] [17] [18] [19]

Included files

The zip contains a ready-to-drop listing module with the feed screen, reusable cards, loading states, empty state, Firestore service helpers, a sample seed script, and a schema guide.

- `ListingCard.jsx` — property image, price, title, locality, chips, verified label, save button.
- `ListingCardSkeleton.jsx` — loading placeholder for initial fetch and refresh.
- `ListingsEmptyState.jsx` — no-results state with reset CTA.
- `ListingsFeedScreen.jsx` — realtime feed with filter chips and pull-to-refresh.
- `services/listings.js` — Firestore subscription and bookmark toggling.
- `utils/seedSampleListings.js` — sample Meerut listings for testing.
- `FIRESTORE_SCHEMA.md` — recommended listing document model and index hints.

What it does

The feed subscribes to active listings in Firestore and updates in realtime when listings are added, edited, or removed, which avoids manual refetch logic for the main home feed. It also supports filters for city, listing mode, property type, and price ranges, which are key details commonly expected in real-estate listing structures. The card layout stays mobile-first and

single-column, which is generally the right default for property-search card interfaces on phones.[20] [18] [21] [19] [22] [16]

Firestore shape

Each property should be one document inside a `listings` collection, with fields like title, listing mode, property type, status, price, bedrooms, bathrooms, area, location, images, and seller info. The included schema file also recommends indexes such as `status + createdAt`, `status + location.city + createdAt`, and `status + price + createdAt`, because filtered Firestore queries often need composite indexes.[17] [16] [20]

Integrated UX

The cards include a cover image, rent/sale badge, save button, metadata chips, and a verified label so users can scan quickly without opening the detail page. Loading is handled by skeleton cards and no-result states use a reset action, which makes the feed feel intentional even before Week 4 polish work.[21]

Next step

The best next move is **Week 2 — Property listing detail page**, because the feed already navigates with a `listingId` param and the detail view is the natural continuation of the listing-card flow. Say "**next**" and I'll build the full **Listing Detail Screen** with image gallery, amenities, seller contact, bookmark button, map/location block, and enquiry CTA.

⁂

1. https://www.telerik.com/blogs/best-practices-for-designing-the-mobile-app-login-screen
2. https://rnfirebase.io/auth/phone-auth
3. https://firebase.google.com/docs/auth/android/phone-auth
4. https://blog.flutterflow.io/phone-authentication-using-flutterflow/
5. https://www.geeksforgeeks.org/android/firebase-authentication-with-phone-number-otp-in-android/
6. https://uxpilot.ai/mobile-design-templates/otp-login-ui
7. https://www.figma.com/community/file/1426916661900143776/authentication-ui-kit-80-screens-sign-in-sign-up-otp-login-forgot-password-etc
8. https://www.nextauth.com/mobile-app-authentication/
9. https://www.youtube.com/watch?v=iTORCXKExAg
10. https://www.figma.com/community/file/1544032395220258224/login-and-otp-screen-design-for-both-mobile-apps-and-responsive-web-applications
11. https://usabilitygeek.com/perfect-mobile-app-login-screen/
12. https://dribbble.com/tags/otp-verification
13. https://dribbble.com/tags/otp
14. https://dribbble.com/search/otp-login

15. https://www.behance.net/search/projects/otp login
16. https://schema.org/RealEstateListing
17. https://firebase.google.com/docs/firestore/data-model
18. https://rnfirebase.io/firestore/usage-with-flatlists
19. https://rnfirebase.io/firestore/usage
20. https://www.pageoptimizer.pro/blog/schema-markup-for-real-estate-a-guide-to-boosting-property-listings-in-search-results
21. https://dcastalia.com/blog/real-estate-website-ui-ux-development/
22. https://www.stan.vision/journal/ui-card-design-examples-best-practices-and-common-patterns
23. https://oneuptime.com/blog/post/2026-03-31-mongodb-design-real-estate-listings-schema/view
24. https://stackoverflow.com/questions/75845834/how-to-get-realtime-data-with-firestore-and-react-native
25. https://www.revivalpixel.com/blog/mobile-first-ui-ux-best-practices-property-search-apps-2026/
26. https://www.scribd.com/document/847641840/Schem-As
27. https://support.google.com/webmasters/thread/27182859/what-schema-should-i-use-for-real-estate-listings-offer-product?hl=en
28. https://rnfirebase.io/database/usage
29. https://github.com/michaelhaar/real-estate-listing-schema
30. https://www.seoclarity.net/blog/structured-data-for-real-estate-listings